Lecture 03.09: Break & Continue (Loop Control Flow)

📌 Executive Summary

Explicit control flow interruption: break, continue, pass statements, and Python's unique loop-else construct.

🔬 Theoretical Foundations & Principles

1. Loop Interruption Semantics

- `break` : Terminates loop execution immediately; control transfers to the next instruction after the loop block.
- `continue` : Aborts current iteration immediately; jumps to loop re-evaluation step.
- `pass` : Syntactic no-op placeholder.

2. Python's Unique `else` Clause on Loops

A loop's `else` suite executes **if and only if** the loop completed all iterations naturally without encountering a `break` statement.

> $$\text{Exec}(\text{else}) \iff \neg\text{Encountered}(\text{break})$$

💻 Production Implementation & Code

```
# Prime search demonstrating loop-else
def is_prime_search(n: int) -> bool:
    if n < 2:
        return False
    for d in range(2, int(n**0.5) + 1):
        if n % d == 0:
            break  # Divisor found, break out
    else:
        # Executed ONLY if loop completed without break
        return True
    return False
```

```
for num in [2, 7, 9, 13, 25, 29]:
    print(f"Number {num:2d} is Prime? {is_prime_search(num)}")
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Detailed in respective algorithmic operations.
- **Auxiliary Space:** Memory bounds and CPython frame management.